

Specification of team software project

Department of Software Engineering

Faculty of Mathematics and Physics, Charles University

Solvers: Bc. Václav Stibor

Study program: Computer Science - Software and Data Engineering

Project title: SAE-Based Interpretable Neural Steering for Recommendation Systems

Project type: Research project

Supervisor: Mgr. Ladislav Peška, Ph.D.

Consultants: RNDr. Patrik Dokoupil

Ing. Vojtěch Vančura, Ph.D.

Mgr. Martin Spišák

Mgr. Petr Škoda, Ph.D.

1 Introduction

This project is part of the research group led by Mgr. Ladislav Peška, Ph.D. at the Department of Software Engineering, Faculty of Mathematics and Physics, Charles University. The research group focuses on explainable and controllable recommender systems and, among other research directions, works on mechanistic interpretability and its applications to recommender systems within the GAČR 25-16785S project.

Recommender systems have become ubiquitous in digital platforms, yet users typically interact with them through a black box: they receive recommendations but have limited control over *how* those recommendations are generated. While users can provide implicit feedback or explicit preferences, they cannot directly influence the underlying reasoning process of the recommendation model. Although today’s recommendation systems achieve impressive predictive accuracy through deep neural architectures, their internal decision processes remain opaque to end users. This lack of transparency and control leads users to feel disconnected from the recommendation process, resulting in dissatisfaction, mistrust, and reduced engagement.

Addressing this opacity requires methods that can decompose neural representations into concepts users can understand and control. Recent advances in interpretability, particularly Sparse Autoencoders (SAEs), offer such a solution by decomposing neural network representations into sparse, semantically meaningful features that are often human-interpretable. For instance, in a movie recommender, SAE features might correspond to concepts like “*1980s sci-fi*”, “*strong female leads*”, or “*slow-paced cinematography*”. By exposing and making these features controllable, we can create a new paradigm of transparent, steerable recommendations.

This project introduces a steerable recommendation system that integrates SAE-derived feature representations directly into the user interaction loop. Rather than developing novel Machine Learning (ML) architectures, we focus on the Human-Computer Interaction (HCI) challenge: designing effective paradigms that bridge the gap between SAE features and users’ expectations. The proposed SAE Steering plugin allows users to manipulate latent features

of the recommendation model via natural language or direct controls, creating a transparent “*Steering Loop*” that empowers users to iteratively refine recommendations and explore the recommendation space through concept-based navigation.

1.1 Relation to other projects

This project directly extends EasyStudy¹, a framework for rapid deployment of recommender system user studies developed by the supervisor’s research group at Charles University. EasyStudy provides essential infrastructure, including dataset management, preference elicitation modules, baseline algorithms, and evaluation metrics. Our work adds SAE-based steering capabilities as a standalone plugin, creating a unified system that maintains full compatibility with the framework’s architecture while introducing novel interpretable steering mechanisms. The high-level plugin architecture within the EasyStudy framework is illustrated in Figure 1, while the detailed system architecture following the C4 model is presented in Section 2.4.

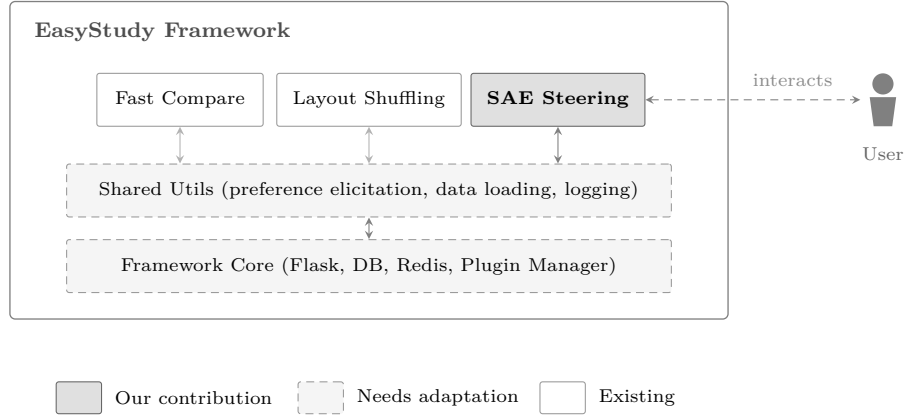


Figure 1: Plugin architecture within the EasyStudy framework. *Our contribution* denotes components developed from scratch; *Needs adaptation* indicates areas requiring substantial modifications to support SAE-based steering; *Existing* components remain largely unchanged but may receive minor adjustments.

The EasyStudy framework employs a plugin-based architecture where study plugins (such as the existing Fast Compare and Layout Shuffling) operate on top of shared utilities and a common framework core. The backend is powered by Python and the Flask micro-framework, with Redis handling session data and caching. The entire environment is orchestrated using Docker, providing reproducible environments for both development and production deployment. The frontend is implemented via Jinja2 templating and JavaScript, allowing for dynamic updates of recommendation lists without full page reloads.

Our contribution will consist of implementing a new SAE Steering plugin that integrates with EasyStudy through the standard blueprint registration mechanism. The plugin will implement a standalone recommendation engine, utilize the existing shared utilities for preference elicitation and data loading, and introduce custom templates and static assets for the steering interface. While the plugin itself represents entirely new development, integration will

¹<https://github.com/pdokoupil/EasyStudy>

require adaptations to the shared utilities layer (e.g., extending preference elicitation for SAE-specific workflows) and potentially minor modifications to the framework core (e.g., configuration handling, caching mechanisms).

Building on foundational research in mechanistic interpretability, particularly work on SAEs as decomposition tools for neural representations, this research direction has been recently initiated by the project supervisor and consultants as part of the GACR 25-16785S project. This project focuses on the interaction design and evaluation aspects that remain unexplored in the existing literature. Our research will complement ongoing work in explainable recommender systems and interactive machine learning, addressing a critical gap: while prior work has focused on post-hoc explanations of recommendations, we investigate whether interpretable internal representations, together with appropriate user interfaces, can serve as effective control mechanisms for preference articulation and refinement.

The project will be linked to the main branch of the EasyStudy repository. During the development phase, we will determine whether to merge the current fork into the main repository or to establish a separate development branch for the extended work. Collaboration with the supervisor and consultants will involve regular meetings to discuss progress, technical challenges, and alignment with the broader research objectives. The consultants will provide expertise in SAE training, recommendation algorithms, and user study design.

2 Project description

A key innovation of this project is the integration of SAE-derived feature representations directly into the user interaction loop. While we leverage existing EasyStudy infrastructure for preference elicitation and data management, the crucial question is how to present SAE features in ways that users can understand and manipulate intuitively. We will develop and implement multiple HCI concepts for feature presentation and steering input methods. Each variant will explore different assumptions about how users conceptualize recommendation criteria and how they prefer to express steering intentions.

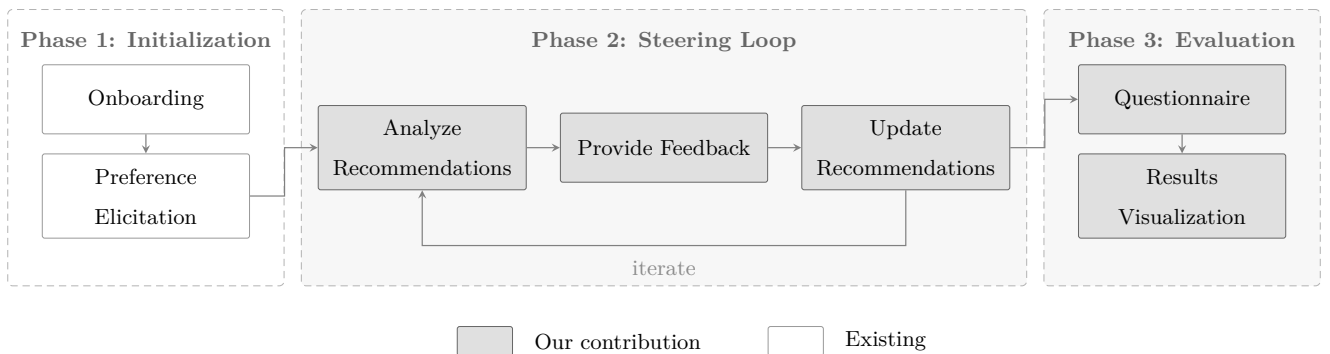


Figure 2: User journey through the EasyStudy with SAE Steering plugin enabled. *Phase 1: Initialization* uses existing components, while *Phase 2: Steering Loop* and *Phase 3: Evaluation* represent our contribution to the iterative refinement and analysis process.

The user journey, illustrated in Figure 2, consists of three distinct phases. The Initialization phase handles onboarding and preference elicitation using existing EasyStudy components, establishing the user’s baseline pref-

erences. The Steering Loop phase represents our core contribution as an iterative cycle where users analyze recommendations, provide feedback through natural language or direct feature manipulation, observe the effects of their steering, and refine until satisfied. This transparent feedback loop empowers users to explore the recommendation space through concept-based navigation rather than opaque algorithmic adjustments. The Evaluation phase captures user feedback through questionnaires and provides results summary visualization, enabling systematic assessment of the steering experience.

The challenge extends beyond User Interface (UI) design to understanding how users interpret features that may not map cleanly to traditional content attributes, and how to translate their steering attempts, which may be ambiguous or contradictory, into meaningful feature adjustments. These HCI experiments will establish the foundation for understanding which interaction paradigms best support user agency in neural recommendation systems.

Beyond the interface layer, the project must address algorithmic considerations for translating user steering intentions into recommendation changes. This includes implementing feature-conditioned reranking mechanisms, exploring latent-space perturbation strategies, and potentially incorporating constrained optimization approaches. While the detailed algorithmic design will evolve during development based on empirical findings, these components are essential for ensuring that user manipulations produce coherent and predictable changes in recommendations.

As part of the EasyStudy integration, we will implement and evaluate multiple interface variants to assess usability and effectiveness. Key variants include:

- **Steering modalities:** sliders for individual SAE features, toggles/checkboxes, example-based steering (prototype selection), and natural-language prompts mapped to feature weights.
- **Presentation modes:** lexical labels with descriptions, prototype items per feature, and visualizations of feature activations.
- **Comparison views:** side-by-side display of baseline versus steered recommendations to provide immediate visual feedback on steering effects.

A final component is questionnaire support tailored to the SAE Steering plugin. The purpose of this component is to enable the evaluation of steering algorithms in follow-up research, where participants provide feedback through questionnaires based on recommendations generated by a selected algorithm at the beginning of a study run. Rather than conducting large-scale comparative studies within this work, the objective is to extend an existing baseline questionnaire module with plugin-specific functionality that supports this study flow.

In the following two subsections, Sections 2.1 and 2.2, we list the functional and non-functional requirements for the overall deployment (EasyStudy with the SAE Steering plugin enabled). For clarification, the term *EasyStudy framework* refers to the existing platform, *SAE Steering plugin* denotes the newly developed EasyStudy plugin, and *overall deployment* refers to their combined deployment. In the list of functional requirements, selected items are labeled with [EXISTING] to highlight requirements related to existing EasyStudy components that the SAE Steering plugin builds upon and may require adaptation.

2.1 Functional requirements

The following functional requirements specify the capabilities of the overall deployment. Requirements labeled [EXISTING] indicate existing EasyStudy functionality that the SAE Steering plugin builds upon and may require adaptation. Requirements are written to be independently implementable and verifiable.

- FR-01: User Onboarding** [EXISTING] The SAE Steering plugin shall guide new participants through an onboarding flow explaining the study purpose and interface controls before preference elicitation begins.
- FR-02: Preference Elicitation** [EXISTING] The SAE Steering plugin shall gather explicit user ratings on specific dataset items to construct the initial user embedding, leveraging existing preference elicitation modules [1].
- FR-03: Dataset Selection** [EXISTING] Researchers shall be able to configure studies to use either MovieLens² and GoodBooks³ as primary domain-specific datasets through existing data loading utilities, with an abstraction layer enabling future extension to other domains (e.g., news, music).
- FR-04: SAE Model Loading** [NEW] The SAE Steering plugin shall load pretrained SAE models and extract interpretable features from user-item interaction embeddings, providing a mapping between the recommendation model’s internal representations and human-understandable SAE feature activations. This process will follow the methodology of decomposing neural representations into monosemantic concepts as established in [2] and its recent application to recommender systems [3] and other state-of-the-art (SOTA) approaches.⁴
- FR-05: Feature Presentation** [NEW] The SAE Steering plugin shall present interpretable features to users through multiple presentation modes: lexical labels with concise descriptions, prototype items representing each feature [7], and visualizations of feature activations.
- FR-06: Multiple Steering Modalities** [NEW] The SAE Steering plugin shall support various steering inputs:
1. Sliders for individual SAE features (bi-directional adjustment)
 2. Toggle switches for feature activation
 3. Example-based steering through prototype selection
 4. Natural-language prompts mapped to feature weights [8]
- FR-07: Toggle-Based Steering** [NEW] Users shall be able to activate or deactivate specific features using toggle switches, applying a default boost or suppression value.
- FR-08: Example-Based Steering** [NEW] Users shall be able to select prototype items to steer recommendations toward similar content. The system shall map item selection to corresponding SAE feature adjustments.

²<https://grouplens.org/datasets/movielens/>

³<https://fastml.com/goodbooks-10k-a-new-dataset-for-book-recommendations/>

⁴This task itself involves significant design decisions such as: (i) which method to use for selecting steerable neuron combinations [4], (ii) how to generate human-readable feature descriptions, leveraging automated LLM-based labeling (e.g., as seen in [5]), (iii) and how to handle features that lack clear semantic interpretations or exhibit low steerability [6].

FR-09: Natural Language Steering [NEW] Users shall be able to submit free-text queries (up to 200 characters) describing desired modifications to recommendation behavior. The SAE Steering plugin shall process the input to extract semantic intent and translate it into actionable steering signals using robust embedding-based NLP techniques [9]. The system shall:

1. Parse user queries using sentence-transformer models for semantic representation.
2. Extract semantic intent and map it to relevant SAE feature dimensions.
3. Correctly interpret linguistic phenomena including:
 - negation (e.g., “no horror”),
 - intensity and emphasis modifiers (e.g., “much more action”)

FR-10: Real-Time Feedback [NEW] When users adjust sliders or enter text queries, the system shall apply user modifications to feature activations and propagate changes through the recommendation pipeline. Supported mechanisms include feature-conditioned reranking, latent-space perturbation [10], and constrained optimization approaches.

FR-11: Side-by-Side Comparison [NEW] The system shall display recommendations from two models simultaneously (Model A and Model B) in horizontally scrollable panels allowing users to directly compare steering effects and understand how their input influences recommendations [11].

FR-12: Steering Reset [NEW] Users shall be able to reset all feature adjustments to default values (0.0) with a single action, restoring baseline recommendations.

FR-13: Iteration History [NEW] The system shall maintain a history of steering iterations within a session, allowing users to view previous feature configurations (last 10 iterations).

FR-14: Session Logging [NEW] The system shall log all user interactions including: timestamps, feature adjustments, text queries, recommendation lists shown, and items clicked. Logs shall be stored in SQLite.

FR-15: Post-Study Questionnaire [NEW] After completing the steering loop, users shall be presented with an extended EasyStudy’s existing baseline questionnaire module with plugin-specific functionality for capturing user experience, preference articulation patterns, and perceived control. Rather than implementing questionnaires from scratch, the objective is to build upon existing survey infrastructure while adding SAE-specific questions and study flow support [12].⁵

FR-16: Results Dashboard [NEW] Researchers shall be able to access a read-only dashboard displaying:

1. Aggregated metrics (participant count, completion rate, mean session duration,...)
2. Feature adjustment distributions across all participants
3. Steering direction ratios (boost, suppress, neutral)

⁵The details to be covered will be determined during development in agreement with the supervisor, as requirements may change over research time.

4. Participant demographics breakdown

The dashboard shall retrieve data from SQLite without performing write operations.

FR-17: Data Export [NEW] Researchers shall be able to export study data in CSV format, including interaction logs, questionnaire responses, and aggregated metrics.

FR-18: Study Configuration [EXISTING] Researchers shall be able to configure study parameters including: active dataset, enabled steering modalities, number of required iterations, and questionnaire items.

2.2 Non-Functional Requirements

The following non-functional requirements specify quality attributes and system constraints. Each requirement is stated in a verifiable and implementation-oriented manner.

NFR-01: Response Time Steering operations (slider adjustments and natural language queries) shall update the recommendation output within 3 seconds under normal load. Initial page load shall complete within 6 seconds.

NFR-02: Concurrent Users The system shall support at least 10 concurrent users without violating the response time constraints defined in NFR-01. The architecture shall allow scaling to 100+ concurrent users by extending Redis-backed session management.

NFR-03: Session Capacity The system shall support at least 500 total user sessions over the study duration, with all interaction data persistently stored for later analysis.

NFR-04: Availability During active study periods, the system shall maintain at least 95% availability, excluding scheduled maintenance windows.

NFR-05: Data Persistence All user interactions (steering actions, recommendation outputs, timestamps, questionnaire responses) shall be persisted within 1 second of occurrence. No interaction data shall be lost during graceful shutdowns.

NFR-06: Reproducibility The entire deployment shall be containerized using Docker and Docker Compose. A new researcher shall be able to deploy a functional local instance within 30 minutes by following the provided documentation.

NFR-07: Maintainability The SAE Steering plugin shall conform to EasyStudy's plugin architecture and maintain clear separation from core framework components.

NFR-08: Extensibility The system architecture shall allow:

- adding a new steering modality by modifying only the plugin module,
- adding a new dataset through configuration and data adapter changes without refactoring the core system.

- NFR-09: Usability** A non-expert user shall be able to complete full steering iteration without external assistance.
- NFR-10: Browser Compatibility** The user interface shall function correctly on current versions of Chrome, Safari, and Edge on desktop devices.
- NFR-11: Researcher Outputs and Observability** The system shall provide researcher-oriented outputs enabling experiment analysis and replication. These shall include exportable logs containing steering actions, feature weights over time, recommendation lists, timestamps, and session identifiers, as well as aggregated summaries suitable for downstream statistical analysis.
- NFR-12: Reliability and Robustness** The system shall handle ambiguous, contradictory, or out-of-scope user input gracefully without runtime failure. In such cases, the system shall provide stable recommendations and record the event for later analysis.
- NFR-13: Documentation** All components, evaluation procedures, and test cases must be thoroughly documented to ensure reproducibility and enable other researchers to build upon the work.

2.3 Mockups

The key components of the user interface enable users to view recommendations from both models side by side for direct comparison. The interface is fully responsive and users can browse recommendations by scrolling or focus on a single model when comparison is not required. In addition, study could be performed just with one model, without the need to compare it with a baseline model. As we mentioned in Section 1.1 the interface is rendered using Jinja2 templates, integrated with EasyStudy’s asset management, while custom JavaScript/CSS handles real-time updates and smooth animations.

The user interface described in Figure 3 employs a split-pane design optimized for comparison. The top section contains two horizontally scrollable recommendation panels, allowing users to cross-reference results from both models. Below lies the *Steering Console*, a fixed panel containing a text input field for natural language steering and a dynamic grid of bi-directional normalized feature sliders. When a user types a query, the sliders automatically animate to reflect the SAE Steering plugin’s interpretation, providing a bidirectional feedback loop.

For the study administrator, a final evaluation read-only dashboard depicted in Figure 4 provides an aggregated view of the research data. This visualization tracks how features were manipulated across multiple participants and quantifies the overall shift in recommendation distributions, enabling researchers to validate the effectiveness of the steering module. This dashboard could be used to analyze the effect of the steering module on the recommendation quality and the user experience. It retrieves all data from SQLite without performing write operations.

2.4 Architecture

This section presents the system architecture using the C4 model notation, covering the first two levels. Additionally, we provide a detailed view of the SAE steering component’s internal data flow.

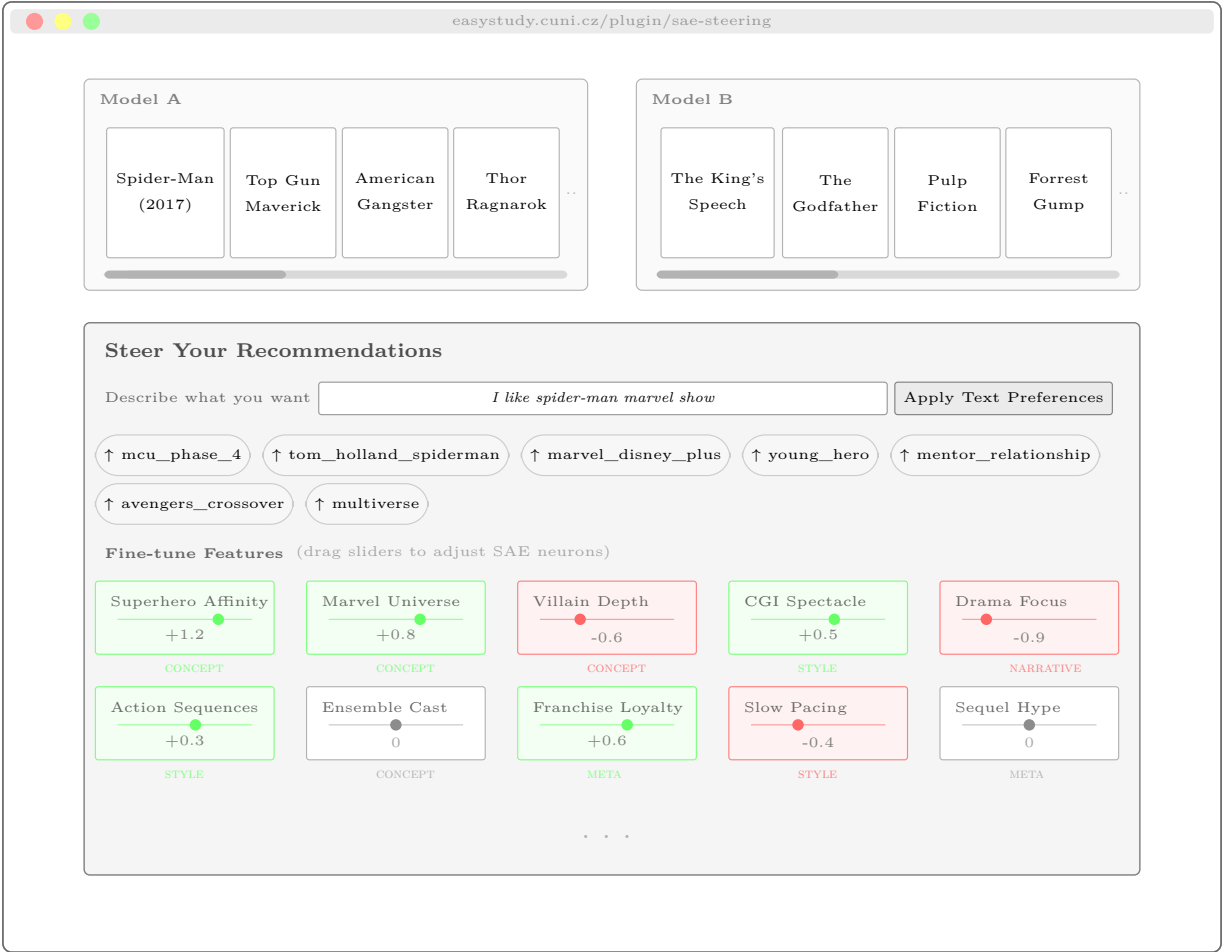


Figure 3: User interface mockup showcasing the split-pane comparison view (top) with baseline and steered model recommendations, and the steering console (bottom) featuring natural language input with automatically suggested SAE feature tags, and fine-tune sliders for direct manipulation of interpretable neural features. Active sliders indicate positive (green) or negative (red) steering adjustments.



Figure 4: Research dashboard mockup for study administrators, displaying key metrics, feature adjustment distributions, and steering patterns analysis. The dashboard operates in **read-only mode**, retrieving all data from SQLite without performing write operations. The card-based layout represents the planned modular structure; specific metrics may evolve based on findings during development.

2.4.1 C4 Level 1: System Context

Figure 5 shows the system context, depicting the EasyStudy platform with the SAE Steering plugin as a unified system and its relationships with external actors and systems.

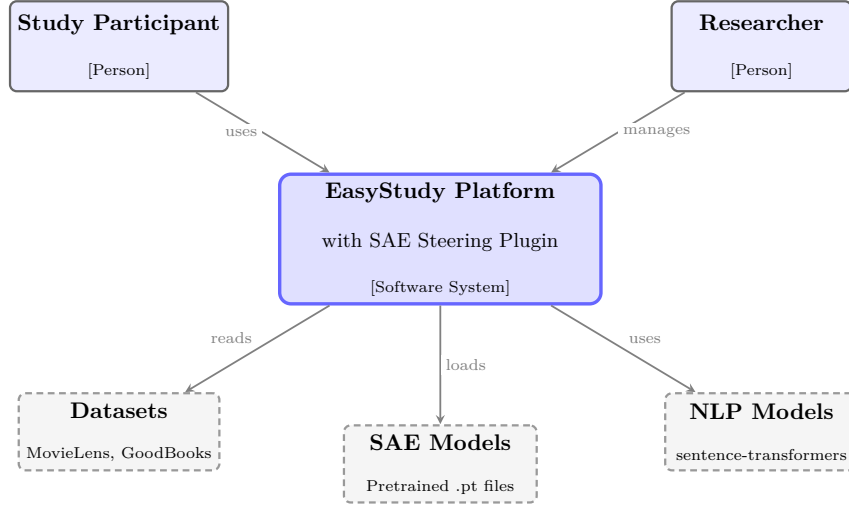


Figure 5: C4 Level 1 – System Context: EasyStudy platform with SAE Steering plugin and its external actors (top) and external dependencies (bottom, dashed).

2.4.2 C4 Level 2: Container Diagram

Figure 6 shows the container diagram, revealing the internal structure of the EasyStudy platform with the SAE Steering plugin. The diagram distinguishes between existing EasyStudy components and our new contributions.

2.4.3 Component Details

Our contribution extends the architecture of EasyStudy’s plugin-based framework with three key new components:

SAE Integration Layer [NEW] Implements interfaces for loading pretrained SAE models and extracting interpretable features from user-item interaction embeddings. The layer handles the mapping between the recommendation model’s internal representations and SAE feature activations. The large tag-neuron index is memory-mapped for efficiency. This layer uses PyTorch for SAE operations, loading models at startup.

Feature Steering Engine [NEW] Develops mechanisms for users to modify feature activations and propagate these changes through the recommendation pipeline. The engine supports both direct feature manipulation (e.g., “increase feature #47 by 30%”) and semantic steering (e.g., “show me more atmospheric films”). The steering vector is computed centrally and applied to Model B while Model A serves as a control group. The engine implements multiple algorithmic options: feature-conditioned reranking (post-hoc), latent-space perturbation with projection, and constrained optimization (soft or hard constraints).

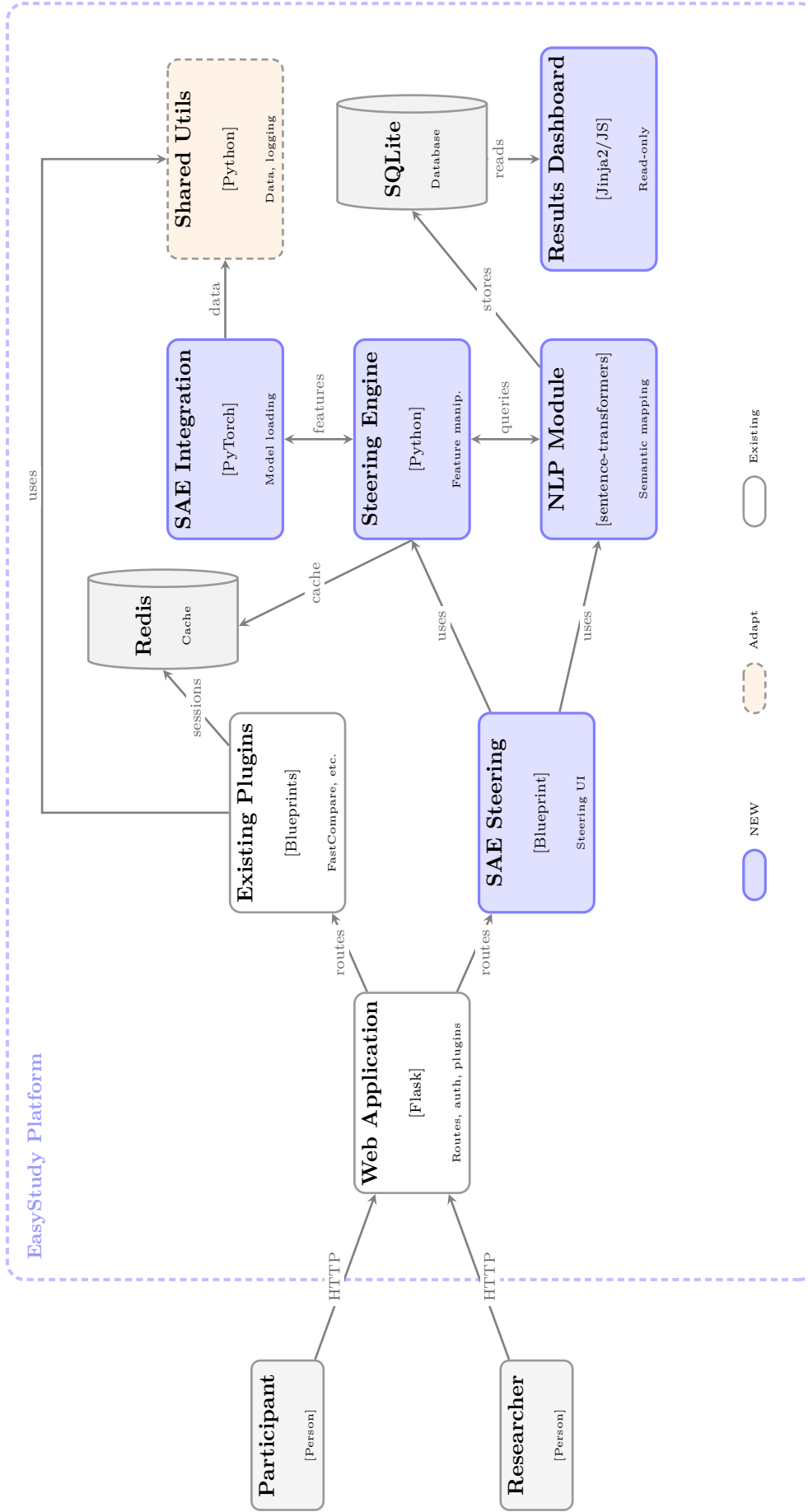


Figure 6: C4 Level 2 – Container Diagram: EasyStudy platform with horizontal data flow. Users (Participants and Researchers) access the Web Application, which routes requests to plugins. The SAE Steering plugin (blue) integrates with the Steering Engine and NLP Module, while the SAE Integration layer handles model loading. Redis provides caching for both existing plugins and the Steering Engine. The Results Dashboard operates in read-only mode, retrieving study data from SQLite. Dashed boxes indicate components requiring adaptation to support SAE-based steering.

User Interface Components [NEW] Design and implement UI elements that present interpretable features to users in accessible ways. This includes feature visualizations (what each feature represents), interactive controls for steering, and explanations connecting features to specific recommendations. The components include custom Jinja2 templates and static CSS/JavaScript files integrated through the plugin’s template and static file directories.

SAERecommender Class [NEW] The core logic encapsulated as a standalone component that provides the `get_recommendations()` method for generating recommendations based on feature adjustments. This class is instantiated and invoked directly from plugin endpoints rather than extending `AlgorithmBase`.

NLP Module [NEW] Uses the sentence-transformers library (specifically the `all-MiniLM-L6-v2` model) for semantic understanding, entity extraction, and sentiment analysis. Maps natural language queries to SAE feature adjustments.

Results Dashboard [NEW] A read-only visualization component that retrieves study data from SQLite without performing any write operations. The dashboard displays aggregated metrics (participant counts, completion rates, mean durations), feature adjustment distributions, steering patterns (boost vs. suppress ratios), and participant demographics and any other relevant metrics that we were possibly find during the development of the plugin as interesting. This separation ensures data integrity and provides researchers with analytical tools for evaluating steering effectiveness.

The existing EasyStudy infrastructure, important for our contribution, components (Shared Utils, Redis, SQLite, etc.) are shown in Figure 6 and remain largely unchanged, requiring only minor adaptations to support SAE-based steering workflows. Note that depending on performance requirements identified during development, a secondary database or cache layer may be introduced for storing precomputed text embeddings and feature descriptions to enable fast semantic lookups without recomputing embeddings at runtime.

2.4.4 Dual-Model Data Flow

The overall deployment follows a dual-model architecture where the steering vector is computed centrally and applied to Model B while Model A serves as a control group, enabling side-by-side comparison of steering effects. Figure 7 illustrates this data flow in detail.

3 Platform and technologies

The project builds upon EasyStudy’s existing technology stack while introducing new dependencies for SAE-based steering. The following technologies are currently in use or planned:

- **Programming Languages:**

- Python – backend logic, SAE operations, recommendation algorithms, and data processing.

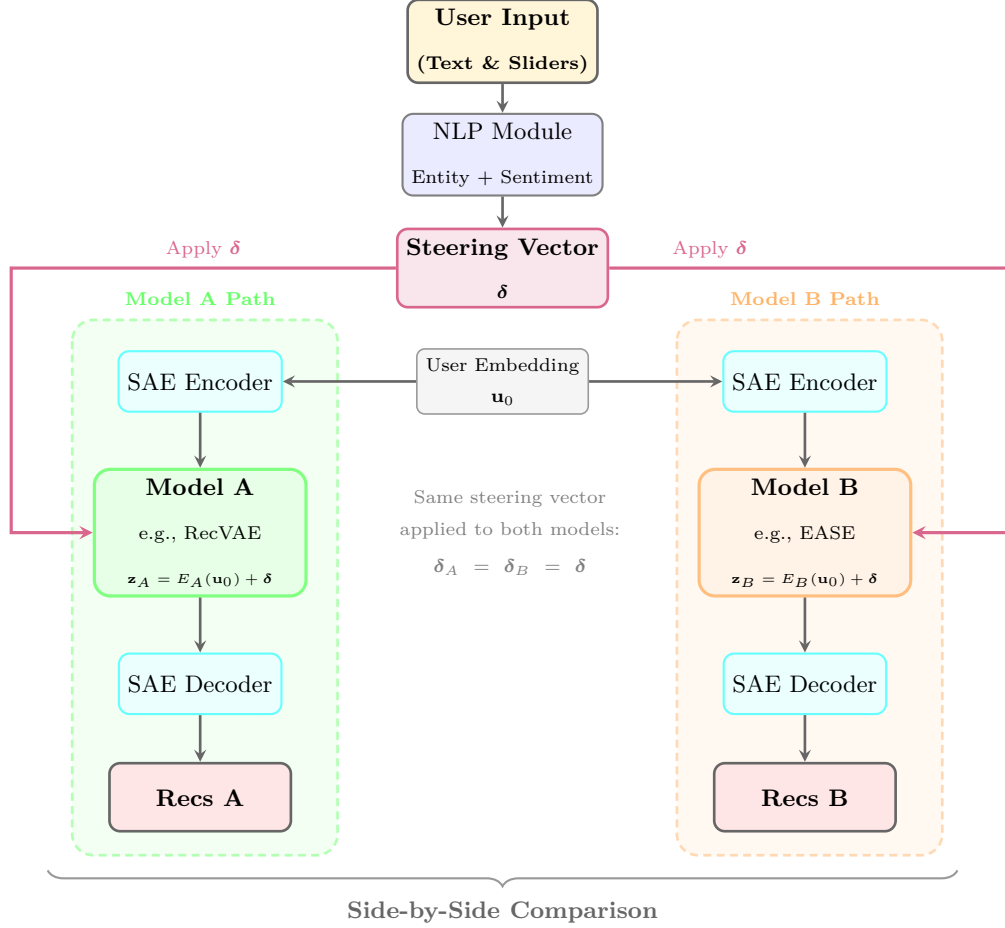


Figure 7: Dual-Model Data Flow: User input is processed through the NLP Module to compute the steering vector δ . Both Model A and Model B receive the same user embedding $\mathbf{u}_0$ and the same steering vector, enabling side-by-side comparison of how different recommendation models respond to identical steering adjustments. Depending on study configuration, either model can operate as a baseline (without steering, $\delta = \mathbf{0}$) or as a steered variant, allowing flexible experimental setups for comparative evaluation.

- JavaScript – frontend interactivity, real-time steering controls, and dynamic UI updates.
- SQL – database queries via SQLite for study and participant data.
- **Frameworks and Libraries:**
 - Flask – micro-framework for the web application and plugin system.
 - Jinja2 – server-side templating for rendering study interfaces.
 - PyTorch – SAE model loading, feature extraction, and neural network computations.
 - sentence-transformers – specifically `all-MiniLM-L6-v2` for mapping natural language to SAE features.
 - NumPy, Pandas – dataset handling and numerical operations.
- **Database and Storage:**
 - SQLite – relational database for studies, users, and interaction logs.
 - Redis – in-memory cache for session management and temporary data.
- **Datasets:**
 - MovieLens – primary dataset for movie recommendations.
 - GoodBooks – secondary dataset for book recommendations.
- **Infrastructure and DevOps:**
 - Docker, Docker Compose – containerization for reproducible deployments.
 - Git – version control and integration with EasyStudy repository.

This list may evolve as implementation details are finalized. New dependencies (PyTorch, sentence-transformers) are integrated within the existing Docker-based deployment pipeline.

4 Evaluation

Our evaluation approach prioritizes demonstrating that SAE-based steering is functionally correct and practically usable, rather than conducting comprehensive comparative studies across interface variants. The primary goal is to show that the concept works and that the interfaces are functional, not to determine which specific design is optimal⁶.

Case-Based Validation The core of our evaluation will consist of systematic tests similar to unit tests for ML systems. We will develop case-based tests that verify steering mechanisms produce expected outcomes for well-defined scenarios. For instance, when a user increases activation of a feature associated with action-oriented content, we should observe measurable and predictable shifts in the genre distribution of recommended items. These tests will include quantitative metrics for steering fidelity, how reliably user manipulations translate

⁶Comparative evaluation of interface variants will be addressed in follow-up research.

into the intended changes in recommendation space, and will verify that different steering modalities correctly propagate user intentions through the entire pipeline.

Steering Fidelity Metrics We will measure how reliably user manipulations translate into intended changes in recommendation space. This includes quantitative metrics such as shift in genre/topic distribution after steering, consistency of steering effects across different user profiles, correlation between feature adjustments and recommendation changes, and accuracy of natural language to feature mapping.

Limited Usability Testing After establishing functional correctness through case-based tests, we will conduct limited usability testing with a small group of participants (5–10 users). The purpose is not to run a full comparative study, but rather to verify that non-expert users can understand and operate the interfaces, and to gather qualitative feedback on intuitiveness and potential improvements. This will help identify any critical usability issues and provide preliminary evidence that the steering concept is accessible to end users.

Questionnaire Infrastructure The SAE Steering plugin includes comprehensive questionnaire modules for capturing user experience, preference articulation patterns, and perceived control. These questionnaires will support validation cases verifying steering correctness and enable future comparative studies.

Documentation for Reproducibility All test cases, evaluation procedures, and results will be thoroughly documented to ensure reproducibility and enable other researchers to build upon our work. The documentation will support integration of our testing framework into the broader EasyStudy ecosystem.

5 Risks

SAE Feature Interpretability SAE features may not map cleanly to user-understandable concepts, making steering less intuitive. We mitigate this by preliminary analysis of feature semantics using prototype items and lexical labels, focusing on the most interpretable subset. If direct manipulation proves ineffective, we fall back to example-based steering.

Natural Language Mapping The semantic mapping from queries to SAE features may be imprecise. We address this through comprehensive NLP testing and bidirectional slider feedback for transparency. If unreliable, we prioritize direct feature manipulation via sliders.

Steering Coherence Ambiguous or contradictory user inputs may produce incoherent recommendations. Constraint validation and visual feedback about conflicts will help; alternatively, UI design can guide users toward consistent patterns.

Performance Real-time steering may cause UI lag. Redis caching and asynchronous processing mitigate this; if needed, we switch to on-demand refresh instead of real-time updates.

Integration Complexity EasyStudy extensions may conflict with future updates. We follow plugin architecture patterns strictly and maintain clear separation. If integration proves too complex, we maintain a separate fork.

Scope Creep Multiple interface variants may exceed available time. We prioritize core modalities (sliders, basic reranker, MovieLens) and focus on feasibility demonstration. If critical, we reduce to a single well-implemented modality.

6 Milestones / Deliverables

By the project’s conclusion, we aim to deliver several concrete outcomes:

Functional EasyStudy Extension A fully functional extension to EasyStudy that integrates SAE-based steering capabilities, including implementation of multiple steering interfaces (sliders, natural language input, example-based steering), SAE integration layer for feature extraction, feature steering engine with multiple algorithmic options, side-by-side model comparison view, and questionnaire infrastructure for evaluation. Each interface variant will be validated to demonstrate essential correctness in how it translates user interactions into steering operations.

Technical Report A comprehensive technical report documenting implemented interface variants and their basic correctness, findings from case-based testing, steering fidelity metrics across different modalities, results from limited user validation, and identification of which steering modality appears most generalizable. The report will be structured to serve either as a standalone demonstration and comparison of the implemented interfaces or as the empirical foundation for a research paper.

Documentation Thorough documentation of all components, evaluation procedures, and test cases to ensure reproducibility and enable other researchers to build upon our work. This includes code documentation and API references, setup and deployment instructions, test case descriptions and expected outcomes, and integration guidelines for the EasyStudy ecosystem.

Demo A live demonstration of the steering system will be presented to the commission, showcasing user interaction flow through the Steering Loop, natural language steering capabilities, direct feature manipulation via sliders, side-by-side comparison of baseline vs. steered recommendations, and real-time visual feedback mechanisms.

Source Code Access The project will be linked to the main branch of the EasyStudy repository (or a separate development branch, to be determined during development). The code will be open-source, accessible through the EasyStudy repository, ensuring reproducibility and enabling future research extensions. The deployment process will follow the same continuous integration and Docker-based pipeline already used in the repository.

The findings and technical contributions will be documented in a form that enables future research papers, establishing the foundational infrastructure and validation methodology necessary for conducting comparative studies of recommendation systems with SAE-based steering.

7 Time Schedule

The project spans October 2025 to May 2026. Work began in October with familiarization of the existing EasyStudy architecture and a proof-of-concept (PoC) integrating SAE models. As of late December 2025, the core prototype and NLP module are in progress. The schedule follows an iterative approach: early prototyping validates core concepts, with subsequent features prioritized based on pilot results. Regular meetings with the supervisor and consultants ensure alignment with research objectives.

Given the exploratory HCI aspects, we anticipate iterations on interface designs based on usability feedback. The schedule reserves adequate time for documentation and deployment, both following EasyStudy’s existing Docker-based pipeline.

Milestone	Activity	Time	Est. MD
PoC & SAE Integration	Framework setup, SAE model integration, feature extraction (foundation for subsequent work)	Oct 2025	11
Prototype	Core steering engine, basic UI (sliders), MovieLens integration	Nov 2025	15
	Testing & validation	Nov 2025	4
NLP Module	Natural language processing, semantic mapping	Dec 2025	11
UI Enhancement	Comparison view, visualizations, questionnaire extension	Jan 2026	12
Results Dashboard	Read-only dashboard, aggregated metrics, feature visualizations	Jan–Feb 2026	6
Extended Features	Additional modalities, GoodBooks dataset, optimization	Feb 2026	12
Case-Based Testing	Test cases, steering fidelity metrics, validation	Mar 2026	11
Limited User Study	Usability testing (5–10 users), feedback collection	Mar–Apr 2026	8
Documentation	Technical report, code documentation, deployment guides	Apr 2026	9
Deployment	Docker setup, CI/CD, repository management	Apr 2026	2
Finalization	Report, demo preparation, presentation materials	May 2026	4

Table 1: Project timeline and estimated effort in man-days (total 105 MD).

Risk buffers: Later months include buffer time to accommodate iterations based on pilot results and technical challenges. If complexity exceeds expectations, we prioritize core functionality (sliders, basic reranker, MovieLens)

and defer extended features.

8 Form of collaboration

Collaboration with the supervisor, Mgr. Ladislav Peška, Ph.D., and the consultants (RNDr. Patrik Dokoupil, Ing. Vojtěch Vančura, Ph.D., Mgr. Martin Spišák, Mgr. Petr Škoda, Ph.D.) will be essential for aligning the project with ongoing research objectives, ensuring technical correctness, and maintaining compatibility with the EasyStudy framework.

8.1 Consulting plan

Regular Meetings Bi-weekly meetings (approximately every 2 weeks) with the supervisor and the research group, each lasting 60–90 minutes. Each participant presents their updates: what they worked on, current status, and planned next steps. This is followed by discussion where others provide feedback and suggestions. This format is standard practice within the group and ensures alignment across related projects.

Consultants Sessions with supervisor or consultants occur in weeks between regular meetings, maintaining roughly weekly contact. These sessions focus on specific and detailed discussion of current technical challenges. Primary collaboration is with Mgr. Ladislav Peška, Ph.D. and RNDr. Patrik Dokoupil. For SAE-specific topics, Ing. Vojtěch Vančura, Ph.D. and Mgr. Martin Spišák provide key expertise in SAE training, feature interpretability, and neural representation analysis. Mgr. Petr Škoda, Ph.D. contributes to deployment aspects.

Ad-hoc Communication Email and messaging for quick questions, code reviews, and urgent technical issues. The project will use the EasyStudy repository’s issue tracking system for bug reports and feature discussions.

8.2 Collaboration with other team members

EasyStudy Authors Mgr. Ladislav Peška, Ph.D. and RNDr. Patrik Dokoupil are the original authors of EasyStudy, providing direct guidance on framework internals, architecture decisions, and integration strategies. Their involvement ensures compatibility and alignment with the framework’s design philosophy.

Domain Experts Other consultants contribute from complementary perspectives. While their EasyStudy experience is primarily as users, they work on similar experimental frameworks and bring valuable expertise: Ing. Vojtěch Vančura, Ph.D. advises on neural representation and SAE methodology, Mgr. Martin Spišák on recommendation algorithms and evaluation, and Mgr. Petr Škoda, Ph.D. in general on deployment. Each perspective enriches the project with insights from different research angles.

GACR 25-16785S Project Team Coordination with researchers working on related SAE and interpretability aspects to share insights, align on evaluation methodologies, and contribute to broader research objectives.

Repository Management The project will be linked to the EasyStudy repository. During development, we will determine whether to merge into the main branch or maintain a separate development branch. Regular commits and pull requests will facilitate collaboration and code review.

References

- [1] Bart P Knijnenburg et al. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction*, 2012.
- [2] Trenton Bricken, Adly Templeton, Joshua Batson, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Anthropic*, 2023. URL <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [3] Xinyi Lin et al. Recsae: Interpret the internal states of recommendation model with sparse autoencoder. *arXiv preprint arXiv:2411.06112*, 2024.
- [4] Nishant Subramani et al. Extracting latent steering vectors from pretrained language models. *arXiv preprint arXiv:2205.05124*, 2022.
- [5] Hoagy Cunningham, Logan Eager, et al. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- [6] James Jardine et al. Interpretable and steerable concept bottleneck sparse autoencoders. *arXiv preprint arXiv:2412.04000*, 2024.
- [7] Steven Bills et al. Language models can explain neurons in language models. *OpenAI blog*, 2023.
- [8] Prasanna Radhakrishnan et al. Steerable reinforcement learning from human feedback. *arXiv preprint arXiv:2312.04390*, 2023.
- [9] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 EMNLP-IJCNLP*, 2019.
- [10] Alex Turner et al. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.
- [11] Sean M McNee et al. Making recommendations better: an analytic model for human-recommender interaction. In *CHI '06 extended abstracts on Human factors in computing systems*, 2006.
- [12] Pearl Pu, Li Chen, and Rong Hu. A user-centric evaluation framework for recommender systems. *Proceedings of the fifth ACM conference on Recommender systems*, 2011.